ZENIKA · BOOTCAMP AI-DRIVEN DEVELOPMENT · MODULE 1

# AI-Driven Development

*Module 1 : Bases du développement agentique*

Mai 2026

# Sommaire


1 Comprendre & cadrer le développement assisté par IA

2 Générer une application (greenfield)

3 Travailler sur une grande application (brownfield)

4 Intégrations tierces avec MCP


© ZENIKA 2026 | All rights reserved

Partie 1

# Comprendre & cadrer le développement assisté par IA




© ZENIKA 2026 | All rights reserved

# Objectifs du module

Ce que vous saurez faire à l'issue de cette journée :

1



### Cadrer

Cadrer un assistant avec des instructions custom (CLAUDE.md) et raisonner en mode plan avant de générer.

2



### Générer et faire évoluer

Générer une application from scratch (greenfield) étape par étape, puis faire évoluer un code existant (brownfield) en respectant ses conventions.

3



### Outiller

Outiller votre IA avec des skills et des agents réutilisables, puis brancher des outils tiers via MCP.

4



### Rester responsable

Produire du code proche de ce que vous écririez à la main, et dont vous restez responsable.

*i* **Takeaway** : l'IA amplifie le développeur, elle ne le remplace pas. C'est vous qui pilotez.

© ZENIKA 2026 | All rights reserved

# Qu’est-ce que l’AI-Driven Development ?

Un mode de travail où le développeur s’appuie sur des **agents IA** (Claude Code, GitHub Copilot…) plutôt que d’écrire chaque ligne à la main.

- On **évite d’écrire le code manuellement** quand c’est possible.
- Le code généré doit être **aussi proche que possible** de celui qu’on aurait écrit :
  - beaucoup de **guidelines** sur la façon de générer,
  - les **meilleurs modèles**,
  - une **relecture systématique** du code produit.
- Le développeur **reste responsable** du code généré.

© ZENIKA 2026 | All rights reserved

# Vibe coding vs AI-driven dev

| | Vibe coding | AI-driven development |
|---|---|---|
| Public | Product Managers, designers UX/UI… | Ingénieurs logiciel |
| Code | Caché par défaut | L'IDE reste l'interface ; le dev maîtrise le code |
| Portée | Prototypage rapide d'applis neuves | Toute appli (legacy, neuve, petite ou grosse…) |
| Outils | Replit, Lovable, Bolt | Claude Code, GitHub Copilot, Cursor, Windsurf, Antigravity |

*i* Ici, on fait de l'**AI-driven dev** : le code est visible, on travaille aussi sur du legacy, et on relit.

© ZENIKA 2026 | All rights reserved

# Une brève histoire



## 2022

GitHub Copilot : autocomplétion dans l'IDE, fonctions et popularité limitées.



## 2023

ChatGPT se popularise : les devs basculent vers le « Shadow AI » pour coder.



## Fin 2024

Cursor rebat les cartes : forte émulation entre Cursor, Copilot, Windsurf ; les outils deviennent agnostiques du modèle.

© ZENIKA 2026 | All rights reserved

# Une brève histoire (suite)



### Fév. 2025

Claude Code : agent en CLI, excellent mode plan (idéal pour le refactoring).



### Fin 2025

Copilot, Cursor, Windsurf sortent tous un mode plan et une CLI.



### Nov. 2025

Google revient dans la course avec Gemini 3 Pro, l'un des meilleurs modèles.

*i* Le paysage bouge vite : on apprend des **techniques**, pas un outil figé.

© ZENIKA 2026 | All rights reserved

# Quel outil pour l'AI-driven dev ?

Les outils **convergent** : les mêmes techniques s'appliquent partout.

`GitHub Copilot` `Cursor` `Windsurf` `Claude Code` `Google Antigravity`

- Cette formation est **volontairement agnostique** dans son discours.
- Les pas-à-pas s'appuient sur **Claude Code** (et une version symétrique existe pour **GitHub Copilot**).
- Copilot est un **fast-follower** très répandu (Microsoft), surtout sur **VS Code** ; le support JetBrains reste instable.

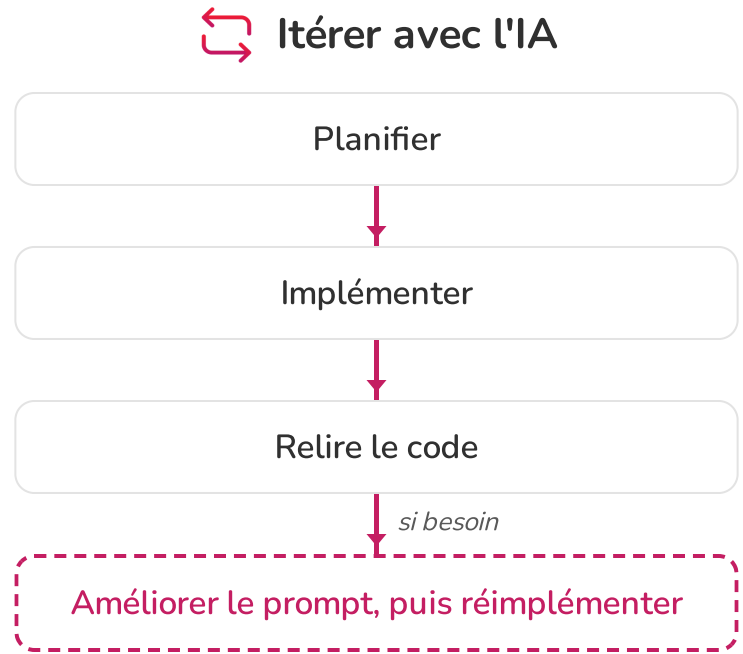
> *i* Développeurs Java : VS Code pour la génération, IntelliJ gardé en parallèle pour le refactoring / debug.

© ZENIKA 2026 | All rights reserved

# Le développeur comme orchestrateur

Avec l'assistance IA, le rôle du développeur se déplace :

- d'écrire chaque ligne → cadrer, diriger, relire,
- de mémoriser la syntaxe → exprimer clairement son intention,
- de chercher sur Stack Overflow → avoir une conversation.




© ZENIKA 2026 | All rights reserved

# Planifier avant de coder

La plus grosse erreur en code assisté par IA : **se jeter directement dans la génération**.

À la place :

1. **Définir le problème** clairement.
2. Le **découper** en tâches plus petites.
3. **Identifier les contraintes** (API, sécurité, performance).
4. **Prototyper mentalement** avant de prompter.
5. **Puis** générer.

> *i* Tous les assistants modernes ont désormais un **mode plan** pour vous aider à cette étape.

© ZENIKA 2026 | All rights reserved

# Relire le code généré

Traitez le code de l'IA comme celui d'un **contributeur inconnu** :

- **Correction** : résout-il vraiment le problème ?
- **Cas limites** : entrée vide, `null`, grandes valeurs ?
- **Sécurité** : entrées validées ? autorisations correctes ?
- **Performance** : N+1, boucles infinies, fuites mémoire ?
- **Lisibilité** : compréhensible dans 6 mois par l'équipe ?
- **Dépendances** : les imports existent-ils vraiment et sont-ils à jour ?

> *i* Faites relire l'IA par elle-même : *« Relis le code que tu viens d'écrire : cas limites, sécurité, perf. »*, étonnamment efficace.

© ZENIKA 2026 | All rights reserved

# Gérer le contexte efficacement

Les modèles ont une **context window** limitée. Sur un gros code, il faut une stratégie :

- **Être sélectif** : n'inclure que les fichiers / extraits pertinents.
- **Résumer** : *« voici notre modèle de données (résumé) »* en arrière-plan.
- **Mentionner** les fichiers utiles dans le prompt.
- **Découper** les grosses tâches en conversations courtes et focalisées.
- Utiliser des **fichiers d'instructions** pour le contexte persistant.

© ZENIKA 2026 | All rights reserved

# Gérer le contexte : un prompt ciblé

## À éviter

**TOUT LE CODE**

*« Corrige mon application » + coller tout le code*

## À la place

**LE BON EXTRAIT**

*« Corrige la fonction calculateDiscount dans pricing.service.ts. Elle reçoit un Cart et un PromoCode, et ignore aujourd'hui le montant minimum d'achat. »*

Donner au modèle **exactement ce dont il a besoin** : le fichier, la fonction, les entrées, le défaut constaté.

© ZENIKA 2026 | All rights reserved

# Claude Code : l'agent dans le terminal

**Claude Code** est l'agent de code d'Anthropic. Il tourne dans le **terminal, l'IDE, le desktop, le navigateur, et la CI/CD**.

- Lit des fichiers, lance des commandes, édite le code, gère Git.
- Toutes les surfaces partagent le **même moteur** : `CLAUDE.md`, skills et hooks fonctionnent partout.
- **Agentique de bout en bout**, dès le départ.

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Lancer dans un projet
cd mon-projet && claude
```

© ZENIKA 2026 | All rights reserved

# Modes de permission

Contrôler **ce que Claude peut faire** sans demander. On bascule avec **Shift+Tab**.

| Mode | Ce qui demande une validation |
|---|---|
| default | Tout (fichiers, commandes) |
| acceptEdits | Commandes seulement : les éditions de fichiers sont auto-acceptées |
| plan | Lecture seule : propose un plan, aucune édition |
| auto *(Teams / Enterprise)* | Un classifieur vérifie chaque action, interruptions minimales |

> *i* Commencez en **plan**, validez, puis passez en **acceptEdits** pour exécuter.

© ZENIKA 2026 | All rights reserved

# Le mode plan : réfléchir avant d'agir

**Mode plan** = Claude explore, raisonne et propose, **sans toucher au moindre fichier**.

```
/plan Je veux ajouter un contrôle d'accès par rôle à l'API.
      Analyse l'authentification actuelle et propose un plan complet.
```

- Claude lit les fichiers, explore, puis **écrit un plan**.
- Vous **relisez et approuvez** avant la première édition.
- À l'approbation, vous choisissez : **auto** / **acceptEdits** / relecture manuelle.

> *i* Bonne pratique : **toujours** passer par le mode plan pour les **gros changements multi-fichiers**.

© ZENIKA 2026 | All rights reserved

Partie 2

# Générer une application (greenfield)




© ZENIKA 2026 | All rights reserved

# Qu’est-ce que le greenfield ?

**Greenfield** = démarrer un projet **from scratch**, sans contrainte de code existant.

C’est là que l’IA offre le **meilleur retour immédiat** :

- pas de legacy à comprendre,
- pas de patterns imposés (c’est vous qui les définissez),
- l’IA peut **scaffolder** des structures entières,
- fini le **syndrome de la page blanche**.

> *i* Mais le greenfield exige aussi le **plus de discipline** : les mauvaises décisions prises vite s’accumulent vite.

© ZENIKA 2026 | All rights reserved

# Le workflow greenfield

## Poser les fondations

1. Définir : écrire une spécification claire du projet
2. Architecturer : concevoir avec l'aide de l'IA
3. Scaffolder : générer la structure de base

## Construire, feature par feature

4. Implémenter : feature par feature
5. Relire : points de revue réguliers
6. Documenter : générer et maintenir la doc

Feature suivante : retour à l'étape 4

© ZENIKA 2026 | All rights reserved

# `CLAUDE.md` : les instructions custom

`CLAUDE.md` est chargé **au début de chaque session** : des instructions persistantes pour l'IA.

Il prescrit deux choses :

- les **pratiques de code** (stack, conventions, commandes),
- des **informations générales** sur l'appli.

```
# Projet : API E-Commerce
## Stack
- NestJS 10 + TypeScript strict : pas de `any`
- TypeORM + PostgreSQL | Jest pour les tests
## Commandes
- `npm run dev`   : serveur de dev
- `npm test`      : lance les tests
## Conventions
- Pattern repository · DTOs pour req/resp
- Validation Zod
```

> *i* Faites la démo **sans** instructions (l'IA part dans tous les sens), puis **avec** : la différence est flagrante.

© ZENIKA 2026 | All rights reserved

# `CLAUDE.md` : bonnes pratiques

**Taille** : rester **sous ~200 lignes** par fichier ; au-delà, l'adhérence chute.

**Spécificité** : les règles **vérifiables** sont mieux suivies.

| Vague | Concret |
| --- | --- |
| *« Formate proprement »* | *« Indentation 2 espaces, pas de trailing commas »* |
| *« Teste tes changements »* | *« Lance npm test avant chaque commit »* |
| *« Range le code »* | *« Les handlers d'API vont dans src/api/handlers/ »* |

© ZENIKA 2026 | All rights reserved

# `CLAUDE.md` : démarrer et ranger

```
/init    # génère un CLAUDE.md de départ en analysant le code
```

## Portées :

| Fichier | Portée |
|---|---|
| `./CLAUDE.md` | Projet, versionné |
| `~/.claude/CLAUDE.md` | Perso |
| Niveau organisation | Géré par la DSI |

© ZENIKA 2026 | All rights reserved

# Étape 1 : Définir le projet

Avec ou sans IA, écrire une **spécification claire** dans un fichier markdown externe :

```
# Spécification : API Gestion de tâches
## Objectif
Une API REST pour gérer ses tâches personnelles.
## Fonctionnalités
- Inscription + authentification JWT
- CRUD des tâches (titre, description, échéance, priorité, statut)
- Filtrage et recherche
## Stack
- Node.js 20 · NestJS · PostgreSQL + TypeORM · JWT · Jest
## Exigences non-fonctionnelles
- p95 < 200 ms · multi-tenant : chacun ne voit que ses tâches
```



La spec devient le **contexte** de l'IA : meilleure spec ⇒ meilleure génération.

© ZENIKA 2026 | All rights reserved

# Étape 2 : L’architecture en mode plan

Utiliser l’IA pour **concevoir avant de coder** :

```
À partir de cette spécification, propose :
1. La structure de dossiers/modules
   d'un projet NestJS
2. Le schéma de base (entités et relations)
3. Les endpoints de l'API (design REST)
4. Les décisions d'architecture clés
   et leurs compromis

N'écris aucun code. Rédige ta proposition
dans architecture.md.
```

Relire l’architecture **de façon critique** :

- correspond-elle au besoin ?
- manque-t-il l’auth, les logs, la gestion d’erreurs ?
- est-on d’accord avec les compromis ?

*i* L’architecture générée est une **proposition**, pas une décision. C’est vous qui en êtes propriétaire.

© ZENIKA 2026 | All rights reserved

# Étape 3 : Scaffolder le projet

Une fois l'architecture validée :

```
Tu vas scaffolder le projet [projet](project.md) à partir de
notre [architecture](architecture.md) :
- Initialiser le projet et créer la structure
- Mettre en place la configuration
- Ajouter et installer les dépendances nécessaires
- Écrire un premier composant end-to-end « bête »
- Démarrer le projet et vérifier qu'il tourne
- Créer le fichier d'instructions IA

N'implémente pas encore la logique métier.
```



Le scaffolding est l'une des tâches IA à **plus forte valeur** : des heures de boilerplate en quelques minutes.

© ZENIKA 2026 | All rights reserved

# Feature par feature : la boucle

## Préparer

1. Récupérer les détails de la feature
2. Comprendre la feature
3. Planifier avec l'IA

## Livrer, puis feature suivante

4. Implémenter
5. Relire
6. Refactorer avec l'IA

*c'est bon (sinon retour à 4)*

7. Ajouter la doc manquante
8. Mettre à jour la doc projet

© ZENIKA 2026 | All rights reserved

# Étape 4 : Implémenter feature par feature

**Une feature à la fois** :

1. Récupérer les détails de la feature (ex. ticket Jira), avec ou sans IA.
2. Savoir **exactement** comment vous l'auriez développée.
3. Demander à l'IA de **planifier** l'implémentation en détail.
4. Implémenter, puis **relire**.
5. **Refactorer** avec l'IA.
6. Ajouter la **doc IA manquante** là où elle s'est trompée.
7. Mettre à jour la doc projet si besoin, puis **feature suivante**.

> *i* Avancer feature par feature empêche l'IA de filer trop loin et de produire du code non testé et difficile à relire.

© ZENIKA 2026 | All rights reserved

# Documentation-Driven Development

Générer la **documentation avant** l'implémentation, pour clarifier le design :

```
Voici un ticket : {lien ou contenu}
Écris un fichier de documentation pour cette feature :
- Explique le besoin métier avec tes propres mots
- Décris l'implémentation cible (archi, fichiers, décisions clés)
- Liste les questions ouvertes et hypothèses
N'écris aucun code.
```

Bénéfices : **vérifier l'implémentation technique**, repérer les **besoins manquants** avant de coder, et obtenir la **doc** de la feature pour la prochaine passe IA.

© ZENIKA 2026 | All rights reserved

# Créer sa propre skill

Une **skill** emballe un savoir-faire reproductible : un `SKILL.md` (mode d'emploi) + des fichiers de support. L'IA la charge **automatiquement** quand elle correspond à la tâche.

```
# « frontend » doit être listée
/skills
```

```
<!-- .claude/skills/frontend/SKILL.md -->
---
name: frontend
description: À utiliser pour les tâches
  de développement frontend.
---
- Charge d'abord architecture-frontend.md
- Crée un plan avant d'implémenter
- Exécute le plan étape par étape,
  en attendant ma validation
```

**Project vs Skill** : un contexte (« tout sur le client X ») vs une **compétence** reproductible (« comment on crée un endpoint ici »).

© ZENIKA 2026 | All rights reserved

Lab 1 · Greenfield

# Générer une application PetClinic

Implémenter une appli PetClinic full-stack avec Claude Code : backend en mode plan, puis frontend via une skill.

Java/Spring + React/Vite ; dépôt `ai-driven-dev-labs`.
Pas-à-pas complet dans le handout « Lab 1 ».




© ZENIKA 2026 | All rights reserved

# Lab 1 : générer PetClinic (1/2)

LAB 1

1. **Importer le dépôt** dans l'IDE, lancer `claude`, vérifier que `CLAUDE.md` se charge ; démarrer back (`./mvnw spring-boot:run`) et front (`npm run dev`).
2. **En mode plan**, décrire le besoin **Pet** (`id, name, ownerName`) : endpoints **GET** (recherche par id, par nom de pet ou de propriétaire) + règle métier **unicité d'un pet par propriétaire**. Décrire le *quoi*, pas le *comment*. L'archi vient de `architecture-backend.md`.
3. Refuser l'exécution immédiate, itérer le plan, l'**enregistrer** dans `docs/execution-plan-user-story-01.md`.

> *i* Piège : Claude écrit parfois `AUTO_INCREMENT` dans `data.sql` (incompatible) → remplacer par `IDENTITY`.

© ZENIKA 2026 | All rights reserved

# Lab 1 : générer PetClinic (2/2)

LAB 1

4. **Exécuter le plan pas-à-pas** en *acceptEdits* : *« exécute la prochaine instruction non cochée ✓, coche-la une fois faite, et arrête-toi pour validation avant l'étape suivante ».*
5. Lancer le back, tester via **Swagger**, lancer `./mvnw clean test` (tout vert), committer.
6. **Frontend via une skill custom** : créer `.claude/skills/frontend/SKILL.md` (charge `architecture-frontend.md`, plan d'abord, exécution pas-à-pas).
7. Générer le **service Fetch** vers l'API Pet + une **page liste des pets**, ajouter les **tests**, vérifier `npm run dev` puis `npm run test`.

> *i* Piège fréquent : l'import de type manquant, par exemple `import { type Pet, findAll } from './petService';`

© ZENIKA 2026 | All rights reserved

# Pièges en greenfield

## Les pièges

- Sur-scaffolding : générer du code qu'on ne comprend pas.
- Sauter les tests : aller vite sans filet de sécurité.
- Dérive d'architecture : du code qui ne respecte plus le plan.
- Hallucinations de dépendances : des paquets qui n'existent pas.
- Logique métier manquante : techniquement correct, fonctionnellement faux.

## Contre-mesures

- Lire chaque fichier généré.
- Lancer les tests en continu.
- Vérifier que les dépendances existent.
- Tester avec des données réalistes.

*i* Le vrai danger : l'**incompétence cachée**, du code qui a l'air bon mais cache des défauts subtils.

© ZENIKA 2026 | All rights reserved

Partie 3

# Travailler sur une grande application (brownfield)




© ZENIKA 2026 | All rights reserved

# Qu’est-ce que le brownfield ?

**Brownfield** = travailler sur un **code existant**, la réalité de la plupart des devs, la plupart du temps.

Défis que l’IA doit surmonter :

- historique et conventions inconnus,
- logique métier non documentée,
- dette technique et contournements,
- chaînes de dépendances complexes,
- tests limités ou absents.

> *i* 70 à 80 % du temps des devs se passe sur du code existant : l’impact de l’IA en brownfield est **énorme**, mais demande **plus d’investissement en contexte** qu’en greenfield.

© ZENIKA 2026 | All rights reserved

# Comprendre un code inconnu

L'IA accélère l'**onboarding** sur un nouveau projet :

```
Je découvre ce code. En partant du point
d'entrée (main.ts), explique l'architecture
globale : quelles sont les couches ?
Comment une requête va de HTTP à la base
de données ? Quels sont les concepts
métier clés ?
```

Ou sur un fichier précis :

```
Explique ce que fait auth.service.ts
et comment il s'insère dans le reste
du système.
```

*i* Ce qui prenait des jours de lecture peut être ramené à quelques heures.

© ZENIKA 2026 | All rights reserved

# La stratégie des fichiers de contexte

Avant d'attaquer un code inconnu, investir dans des **fichiers de contexte**. Plutôt que de partir de zéro, laisser l'IA analyser d'abord :

```
/init
```

L'agent scanne le projet et **génère** les instructions. Puis **relire de façon critique** (essentiel en brownfield) :

- les concepts métier sont-ils bien identifiés ?
- les **pièges et invariants** sont-ils captés ? *(probablement pas)*
- des patterns implicites manqués ? quelque chose d'halluciné ?

> `/init` donne un **point de départ**. **Votre expertise comble les trous** (surtout la section « pièges connus »).

© ZENIKA 2026 | All rights reserved

# Fichiers de contexte par module

Sur un gros code, ajouter des fichiers d'instructions au niveau module :

```
# src/billing/instructions.md
## Ce module
Gère la facturation : cycle de vie des abonnements, génération de
factures, traitement des webhooks Stripe.
## Règles critiques
- Toutes les méthodes publiques d'InvoiceService doivent être idempotentes
- Ne jamais propager une erreur Stripe brute, l'emballer en BillingException
- Utiliser le value object Money pour tout montant
```

Bénéfices : plus petits et focalisés (l'IA les lit en entier), maintenus par l'équipe propriétaire, sans polluer le contexte global.

> *i* Ces fichiers **doivent** évoluer avec le code.

© ZENIKA 2026 | All rights reserved

# Ajouter une feature en respectant l'existant

La clé : **contraindre l'IA aux patterns existants**.

```
Je dois ajouter un endpoint « bulk delete tasks ».

AVANT d'écrire du code :
1. Regarde le DELETE /tasks/:id existant pour les patterns
2. Regarde comment on fait les opérations en masse ailleurs
3. Vérifie nos patterns de validation et de gestion d'erreurs
4. Puis propose une implémentation cohérente avec le code

N'introduis aucun nouveau pattern ni dépendance.
```

> *i* L'instruction « n'introduis aucun nouveau pattern » est cruciale : l'IA propose souvent des solutions modernes qui ne collent pas au style existant.

© ZENIKA 2026 | All rights reserved

# Écrire des tests sur du legacy

L'une des tâches IA à plus forte valeur en brownfield : **ajouter de la couverture de tests**.

```
Écris des tests unitaires complets pour la classe CartService,
aujourd'hui non testée. Couvre :
- toutes les méthodes publiques
- les cas limites (panier vide, quantités invalides, conflits de remise)
- les cas d'erreur (produit absent, stock insuffisant)
On utilise Jest. Mocke les dépendances (InventoryService, PricingService).
```

Bénéfices : **filet de sécurité** pour le refactoring, **documentation** du comportement attendu, et l'IA est forcée de **comprendre** le code en profondeur.

© ZENIKA 2026 | All rights reserved

# Skills de convention : une skill par pattern

Définir des **skills atomiques** : une par type de tâche récurrente.

| Fichier de skill | Quand l'invoquer |
|---|---|
| `skills/new-endpoint/SKILL.md` | Ajouter un endpoint REST |
| `skills/new-service/SKILL.md` | Créer une classe service |
| `skills/add-tests/SKILL.md` | Écrire des tests |
| `skills/new-migration/SKILL.md` | Migration de base |

Chaque skill encode vos conventions **pour cette tâche précise** : nommage, imports requis, patterns à suivre, choses à éviter.

> *i* **Atomique = une préoccupation par fichier.** Une skill qui couvre tout est ignorée ; une skill qui couvre une tâche est toujours pertinente.

© ZENIKA 2026 | All rights reserved

# Les agents (sous-agents)

Les **sous-agents** sont des instances Claude isolées, avec leur **propre context window** et leur **jeu de tools**.

- Intégrés : **Explore** (Haiku, lecture seule), **Plan** (sans édition), **general-purpose**.
- Custom : `.claude/agents/<nom>.md`, avec tools choisis, modèle par défaut, system prompt, **mémoire persistante** (`memory: user`).

```
<!-- .claude/agents/code-reviewer.md -->
---
name: code-reviewer
description: Relecteur expert. À utiliser
  après avoir écrit du code.
tools: Read, Grep, Glob, Bash
model: sonnet
---
Tu es un relecteur senior : qualité,
sécurité, bonnes pratiques.
Lance `git diff`, puis classe les retours
en Critique / Avertissement / Suggestion.
```

*i* Le contexte d'un sous-agent est **séparé** : l'exploration (logs, grep) ne pollue pas la conversation principale ; seul le résumé revient.

© ZENIKA 2026 | All rights reserved

# Refactorer avec l'IA : en sécurité

Refactorer = **transformer le code sans changer son comportement**. L'IA y excelle, **à condition d'avoir des tests**.

1. **Identifier** : trouver le code à refactorer.
2. **Tester** : s'assurer qu'il est testé (générer si besoin).
3. **Planifier** : décrire l'état cible, faire planifier l'IA.
4. **Transformer** : l'IA applique la transformation.
5. **Vérifier** : lancer les tests, tous doivent passer.
6. **Relire** : vérifier que le résultat a du sens.

> *i* Ne **jamais** sauter l'étape 2 (tests) ni l'étape 5 (vérifier). Sans tests, le refactoring n'est que de l'espoir.

© ZENIKA 2026 | All rights reserved

# Patterns de refactoring

Nommez le **pattern précis**. L'IA les connaît et les applique fidèlement :

- **Extract Function**, **Replace Conditional with Polymorphism** (Strategy), **Introduce Repository**, principes **SOLID**…
- Migrations : version de framework, librairie (axios → fetch), langage (JS → TS).

> *i* Procéder **par petits pas** vérifiables : un gros refactor en un seul coup échoue.

© ZENIKA 2026 | All rights reserved

# Tests de caractérisation

Avant un gros refactor sans tests, générer des **tests de caractérisation** :

```
Avant de refactorer InvoiceCalculator, génère des tests qui capturent
le comportement ACTUEL, peu importe qu'il soit correct : je veux savoir
si je change quelque chose par accident.
```



Ils ne vérifient pas la correction : ils **détectent le changement**.

© ZENIKA 2026 | All rights reserved

Lab 2 · Brownfield

# Refactorer PetClinic

Faire évoluer une PetClinic plus riche (Vets, Visits, Invoices). Refactor back + front, vérification, puis guide utilisateur.

Pas-à-pas complet dans le handout « Lab 2 ».




© ZENIKA 2026 | All rights reserved

# Lab 2 : refactorer PetClinic (1/2)

LAB 2

1. **Mise en place** : ouvrir `brownfield-backend` et `brownfield-frontend`, explorer, lancer back et front.
2. **Créer une skill** `backend` (sur le modèle de `frontend`, à partir de `architecture-backend.md`) ; vérifier que `frontend` et `backend` sont disponibles.
3. **Feature backend** : faire de **Owner une entité à part entière** (au lieu d'un champ de `Pet`) : `id, name, address` ; API toujours exposée via Pet et Visit. Planifier, **inclure le refactoring des tests** (souvent oublié), implémenter, tests verts, tester sur Swagger.

> *i* Le brownfield est plus dur que le greenfield : code plus gros ⇒ **plus de guidage**.

© ZENIKA 2026 | All rights reserved

# Lab 2 : refactorer PetClinic (2/2)

LAB 2

4. **Feature frontend** : depuis la liste, **cliquer un pet affiche ses détails + ceux du propriétaire**. Planifier, implémenter, vérifier dans le navigateur.
5. **Générer un guide utilisateur** : créer une skill `documentation`.
   - Le guide doit rester **concis**.
   - La skill **ne lit pas le code** : elle **parcourt l'application qui tourne** et décrit comment l'utiliser.
6. Invoquer la skill sur l'URL du front ( `http://localhost:5173/` ) pour remplir `docs/user-guide/user-guide.md`, relire et itérer.

> *i* Les captures d'écran du guide seront ajoutées au lab MCP, avec Playwright.

© ZENIKA 2026 | All rights reserved

Partie 4

# Intégrations tierces avec MCP




© ZENIKA 2026 | All rights reserved

# Au-delà de la génération de texte

Les outils de code modernes vont plus loin que générer du texte, ils **utilisent des tools** :

- **lire et écrire des fichiers**,
- **exécuter des commandes shell** et en capturer la sortie,
- **chercher sur le web**,
- **interroger des API** et des bases de données,
- **interagir avec des services** (GitHub, Jira, Slack…),
- **utiliser des outils spécialisés** (analyse de code, scanners de sécurité).

> *i* C'est ce qui transforme l'IA d'un **générateur de texte** en **agent**.

© ZENIKA 2026 | All rights reserved

# Qu’est-ce que MCP ?

Le **Model Context Protocol** (MCP) est un standard ouvert créé par **Anthropic** (2024), désormais adopté par l’industrie.

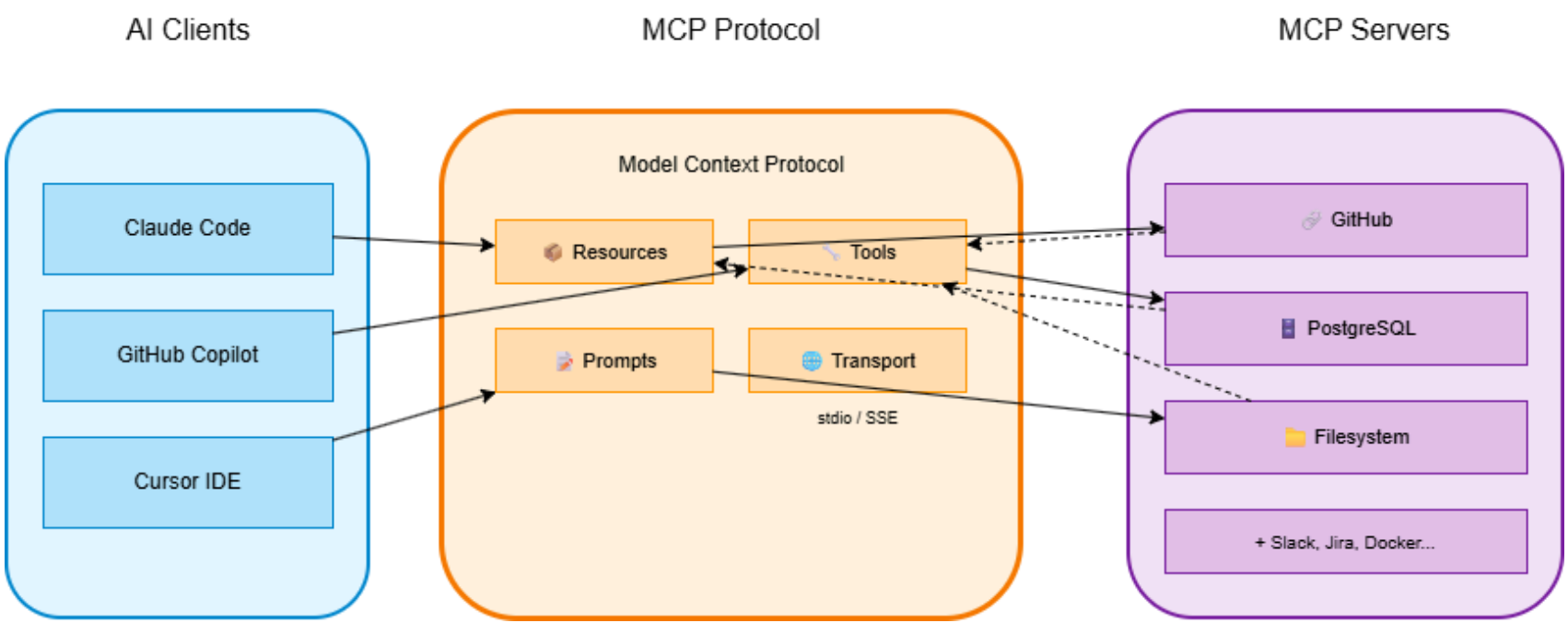
MCP définit :

- une **interface standard** pour qu’une IA dialogue avec des outils externes,
- une architecture **client-serveur** (client IA, serveur d’outils MCP),
- le support des **resources**, **tools** et **prompts**.

> *i* Voyez MCP comme l’**USB des outils IA**. Branchez n’importe quel serveur MCP sur n’importe quel client compatible.

© ZENIKA 2026 | All rights reserved

# Architecture MCP




© ZENIKA 2026 | All rights reserved

# Serveurs MCP populaires

| Serveur MCP | Ce qu'il débloque |
|---|---|
| GitHub / GitLab | Lire des issues, créer des PR, gérer les branches |
| Jira | Lister les sprints, créer / modifier des tickets |
| Figma | Lire les design tokens, inspecter les composants |
| Context7 | Docs de librairies à jour injectées dans le contexte |
| SQL (PostgreSQL, MySQL…) | Interroger les schémas, analyser les requêtes lentes |



Les serveurs MCP sont publiés en paquets npm, à installer à la demande avec `npx -y`.

© ZENIKA 2026 | All rights reserved

# Configurer MCP dans Claude Code

Dans `.mcp.json` (projet) ou `~/.claude/claude.json` (global) :

```
{
  "mcpServers": {
    "figma": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "figma-developer-mcp",
        "--figma-api-key=...", "--stdio"]
    }
  }
}
```

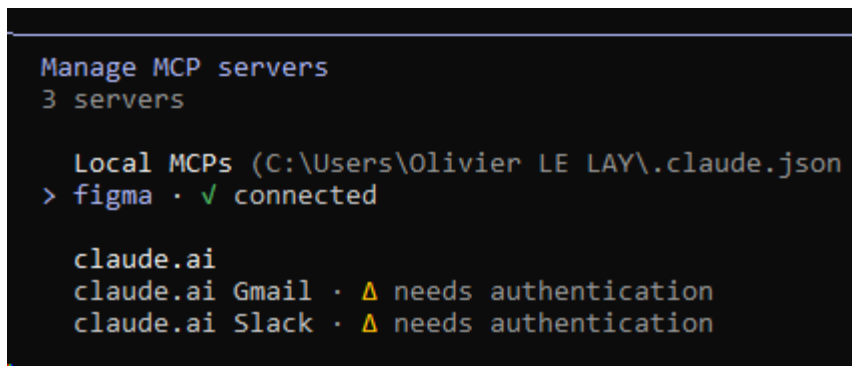
En ligne de commande :

```
# écrit .mcp.json
claude mcp add figma -- \
  npx -y figma-developer-mcp --stdio
# lister les serveurs
claude mcp list
claude mcp remove figma
```

*i* Ne jamais coder en dur une clé d'API : variables d'environnement ou gestionnaire de secrets.

© ZENIKA 2026 | All rights reserved

# Activer un serveur MCP




Activer / désactiver les serveurs MCP dans le chat avec `/mcp`.

© ZENIKA 2026 | All rights reserved

# MCP en action : Figma → composant

Regarde le composant « card » dans mon fichier Figma https://figma.com/design/abc123/...node-id=12:34 et génère le composant qui correspond.

*[récupère le nœud Figma via MCP] Je vois une card : image, titre, sous-titre, bouton CTA en couleur primaire #1A73E8. Je génère ProductCardComponent... Espacement (16px padding, 8px gap) et typo (18px/600, 14px/400) repris directement des design tokens.*

*i* Le serveur Figma expose tokens, contraintes de layout et propriétés des composants. L'IA les lit directement, sans copier-coller depuis l'inspecteur.

© ZENIKA 2026 | All rights reserved

# MCP en action : GitHub (issue → PR)

**Regarde les issues « bug » ouvertes du repo, prends la plus simple, crée une branche, implémente le fix, lance les tests et ouvre une pull request.**

*[lit les issues via MCP] 3 bugs ouverts. Le plus simple : #234 « ordre de tri perdu après filtrage ». Je crée la branche fix/sort-order-filtering… [lit le test rouge, implémente le fix] Tests verts. PR #267 ouverte : https://github.com/org/repo/pull/267*

*i* Ce flux **issue → PR** fonctionne aujourd'hui avec Claude Code + serveur GitHub.

© ZENIKA 2026 | All rights reserved

# Choisir ses MCP avec parcimonie

**Chaque MCP, c'est du contexte.** Chaque serveur activé ajoute :

- le nom de **tous ses tools** (resources, prompts),
- des bouts de documentation,
- des variables d'environnement et détails de config.

Activer 5 MCP = **des milliers de tokens** injectés à **chaque** requête.

> *i* N'activez que les MCP que vous **utilisez vraiment** pour ce projet.

© ZENIKA 2026 | All rights reserved

# Alternative : CLI + Skills

Tout n'a pas besoin d'un MCP. Les outils classiques marchent très bien :

```
# CLI GitHub/GitLab
brew install glab
npm install -g jira-cli-node
```

… puis on les **emballe dans une skill** qui décrit *quoi faire* :

```
# ./skills/open-issue/SKILL.md
Créer une issue avec titre et description.
Utiliser `glab issue create --title "..."
  --description "..."`.
```

> *i* MCP = structure automatique (schéma d'API, typage). **CLI + Skills** = contrôle (on n'active que ce qu'on veut) et **coût de contexte nul** quand inutilisé. La plupart des équipes mélangent les deux.

© ZENIKA 2026 | All rights reserved

# Pour aller plus loin : les hooks

Les **hooks** exécutent des scripts à des points clés du workflow, **toujours**, sans décision du LLM.

| Événement | Quand il se déclenche |
|---|---|
| `PreToolUse` | Avant un outil, peut le **bloquer** (exit 2) |
| `PostToolUse` | Après le succès d'un outil |
| `SessionStart` / `Stop` | Début/reprise de session · fin de réponse |



`PreToolUse` peut **bloquer** une action (ex. protéger `.env`) ; `PostToolUse` ne peut pas l'annuler. Bloquer **avant**.

© ZENIKA 2026 | All rights reserved

# Un hook en pratique

Formater avec Prettier chaque fichier édité ou écrit par Claude :

```
{ "hooks": { "PostToolUse": [{
  "matcher": "Edit|Write",
  "hooks": [{ "type": "command",
    "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
  }]
}] } }
```



Le hook tourne à **chaque** `Edit` ou `Write`, sans que le LLM ait à y penser.

© ZENIKA 2026 | All rights reserved

Lab 3 · MCP : Playwright & Figma

# Brancher des outils tiers

Faire utiliser à Claude les serveurs Playwright et Figma pour tester, implémenter un design, et enrichir le guide.

Pas-à-pas complet dans le handout « Lab 3 ».




© ZENIKA 2026 | All rights reserved

# Lab 3 : brancher des outils tiers (1/2)

LAB 3

1. **Playwright** : `claude mcp add playwright npx @playwright/mcp@latest`.
   Avec les apps brownfield lancées, demander un **test exploratoire** sur `http://localhost:5173/` (ouvre un navigateur, **Chrome requis**), puis **générer un script** et le lancer ( `npx playwright test` ).
2. **Figma** : `claude mcp add Framelink_Figma_MCP -- npx -y figma-developer-mcp --figma-api-key=… --stdio`. Via `/mcp`, **désactiver le Figma intégré** (plus limité) pour utiliser Framelink.

> *i* Le MCP Figma se trompe sur icônes/couleurs : relire au plan **et** à la génération, avancer lentement.

© ZENIKA 2026 | All rights reserved

# Lab 3 : brancher des outils tiers (2/2)

LAB 3

3. **Implémenter le design Figma** : créer une skill `frontend-figma` (comme `frontend`, mais avec les outils Figma activés). Faire un **plan d'abord**, puis refactorer **par incréments** : le **Header**, puis chaque page (**Pets, Visits, Vets, Invoices**).
4. **Mettre à jour le script Playwright** : après la refonte, relancer `npx playwright test` ; si ça casse, invoquer la skill Playwright pour **corriger les tests**.
5. **Enrichir le guide utilisateur** : donner à la skill `documentation` l'accès à Playwright, puis régénérer le guide **avec des captures d'écran** de toutes les pages.

> *i* **Bilan** : un agent qui **teste, code depuis un design, et documente**, chacun via une skill dédiée.

© ZENIKA 2026 | All rights reserved

Conclusion

# Synthèse & ouverture




© ZENIKA 2026 | All rights reserved

# Notre workflow agentique

Au fil de la journée, on a construit une **boîte à outils de skills** :

`backend` `frontend` `frontend-figma` `documentation` `playwright` `debug`

- Chaque skill encode **une compétence** réutilisable et relisible.
- Les **agents** isolent les tâches lourdes (revue, exploration, migration).
- Les **MCP** apportent les outils externes directement dans le prompt.

> *i* Ce ne sont que des exemples : votre projet en appellera d'autres.

© ZENIKA 2026 | All rights reserved

# Ce que vous avez appris

1. Pour une bonne génération, il faut des **instructions custom**, des **agents** et des **skills**.
2. **Toujours relire** le code généré : vous en restez responsable.
3. Les **serveurs MCP** apportent vos outils dans le contexte (avec parcimonie).
4. Guide utilisateur et **spécifications produit** se génèrent aussi avec l'IA.
5. On sait **générer** (greenfield) et **faire évoluer** (brownfield) une appli, sur son poste.

© ZENIKA 2026 | All rights reserved

# Les 4 réflexes

1



## Cadrer

Les instructions custom.

2



## Planifier

Le mode plan.

3



## Outiller

Skills, agents, MCP.

4



## Relire

Les tests, le diff.

© ZENIKA 2026 | All rights reserved

# Ouverture : le Spec-Driven Development

On a structuré notre dev **autour du code**. L'étape suivante : faire de la **spécification la source de vérité**.

- Générer des **user stories** et des specs, pas seulement du code.
- Des frameworks dédiés : **OpenSpec**, **Spec Kit**, **BMAD**.
- Garder le code **aligné sur la spec** (anti-drift), industrialiser et **mutualiser en équipe**.

> *i* C'est tout l'objet du **Module 2 : Développer avec l'IA en équipe**. On y reprend ces bases pour aller vers le **Spec-Driven Development** et les **workflows custom**.

© ZENIKA 2026 | All rights reserved

# Des questions ?

© ZENIKA 2026 | All rights reserved